

Use PHONY to avoid file-target name conflicts

Remember the *all* and *clean* 'special' targets in our *Makefile*? What happens when the project directory has files with the names 'all' or 'clean'? The conflicts will cause errors. Use the .PHONY directive to specify which targets are not to be treated as files—for example: *.PHONY: all clean*.

Simulating *make* without actual execution

At times, maybe when developing the *Makefile*, we may want to trace the *make* execution (and view the logged messages) without actually running the actions, which is time consuming. Simply use *make -n* to do a 'dry run'.

Using the shell command output in a variable

Sometimes we need to use the output from one command/action in other places in the *Makefile*—for example, checking versions/locations of installed libraries, or other files required for compilation. We can obtain the shell output using the *shell* command. For example, to return a list of files in the current directory into a variable, we would run: *LS_OUT = \$(shell ls)*.

Nested *Makefiles*

Nested *Makefiles* (which are *Makefiles* in one or more subdirectories that are also executed by running the *make* command in the parent directory) can be useful for building smaller projects as part of a larger project. To do this, we set up a target whose action changes directory to the subdirectory, and invokes *make* again:

```
subtargets:
    cd subdirectory && $(MAKE)
```

Instead of running the *make* command, we used *\$(MAKE)*, an environment variable, to provide flexibility to include arguments. For example, if you were doing a 'dry run' invocation: if we used the *make* command directly for the subdirectory,

the simulation option (*-n*) would not be passed, and the commands in the subdirectory's *Makefile* would actually be executed. To enable use of the *-n* argument, use the *\$(MAKE)* variable.

Now let's improve our original *Makefile* using these advanced features:

```
CC = gcc # Compiler to use
OPTIONS = -O2 -g -Wall -g for debug, -O2 for
optimise and -Wall additional messages
INCLUDES = -I . # Directory for header file
OBJS = main.o module.o # List of objects to be
build
.PHONY: all clean # To declare all, clean are
not files

all: $(OBJS)
    @echo "Building.." # To print "Building.."
message
    $(CC) $(OPTIONS) $(INCLUDES) $(OBJS) -o
target_bin

%: %.c # % pattern wildcard matching
    $(CC) $(OPTIONS) -c $*.c $(INCLUDES)

list:
    @echo $(shell ls) # To print output of
command 'ls'

clean:
    @echo "Cleaning up.."
    -rm -rf *.o # - prefix for ignoring errors
and continue execution
    -rm target_bin
```

Run *make* on the modified *Makefile* and test it; also run *make* with the new *list* target. Observe the output.

Make in non-compilation contexts

I hope you're now well informed about using *make* in a programming context. However, it's also useful in non-programming contexts, due to the basic behaviour of checking the modification timestamps of target files and dependencies, and running the specified actions when required. For example, let's write a *Makefile* that will manage an image store for us, doing *thumbnailing* when required. Our


redhat.
TRAINING PARTNER
RHCE / RHCSS Exam Centre
OpenLDAP+RHDS+ Active Directory
Clustering
Shell Scripting
Solaris 10 * PHP-MySQL
ORACLE
MCTS / MCITP
Windows 2008 Server


100% Placement Assistance

RHCE & RHCSS Training
By RHCSS Faculty
Only at AEM in Kolkata

Offer!! **RHCE & MCSE @ Rs. 18990/-**
MCSA & CCNA @ Rs. 13990/-
RHCSS @ Rs. 12250/-

Students Placed in :
Wipro, HP, IBM, TCS, Tripod,
Comtel, Diadem, PCS, CMS,
RedHat & Many More...


AEM™
Enhancing Knowledge Horizon
ISO 9001:2000

North Kolkata :
2/80, Dumdum Road
Ph. : 9830051236
9830075018
South Kolkata :
7/1, Jadunath Sarkar Road
Ph. : (033) 32518779
9330925622
www.aemk.org